

Box Transpose: In-Place N-Dimensional Permutation on GPUs

Kaveen Jayamanna

[PLACEHOLDER] Affiliation

Abstract

[PLACEHOLDER] Abstract. One paragraph: the problem, the idea, the headline result.

1 Introduction

[PLACEHOLDER] Introduction.

2 Background

[PLACEHOLDER] Permutation as a memory-movement problem; why out-of-place copies are the thing to avoid.

3 Design

3.1 Adjacent box swaps

[PLACEHOLDER] A box is a run of adjacent axes treated as one unit. A swap exchanges two adjacent boxes, and is carried out as an in-place transpose of a logical $rows \times cols$ matrix, batched over D_{pre} and vectorised over D_{post} .

3.2 Planning by shortest path

A permutation can always be reached by bubbling each axis into position with single-axis swaps. Treating a run of adjacent axes as one box instead turns the decomposition into a shortest-path problem over axis orderings, and the search is free to spend one swap where the baseline spends several.

To measure what that freedom is worth we plan every permutation of a tensor of 40,320 elements at each rank from 2 to 6 — five shapes, 208 permutations, each planned by both strategies for 416 plans in all. The comparison is paired: both planners see the same shape and the same permutation, and every rank holds the element count fixed, so a difference in

Table 1: **What the box search saves over the adjacent baseline, by rank.** 208 permutations across 5 ranks, every one of them planned by both strategies on a tensor of the same size. The bytes-moved advantage grows with rank, from 1.0x where a permutation is a single transpose to 2.8x at rank 6.

Rank	Shape	Cases	Swaps		Bytes	Aux (B)
			Adj.	Search		
2	192×210	1	1	1	1.0x	64
3	48×40×21	5	2	1	1.5x	80
4	24×40×6×7	23	3	2	2.0x	480
5	24×8×5×6×7	119	5	2	2.4x	1,064
6	4×6×8×5×6×7	60	7	2.5	2.8x	3,488

Swaps and bytes are medians over the cases in that row; **Bytes** is the median per-case ratio of bytes moved by the adjacent baseline to bytes moved by the search, so higher is better. **Aux (B)** is the largest cycle-metadata footprint any one search plan in the row needed, in bytes. Rank 6 is a seeded sample of its permutations; every other rank is exhaustive.

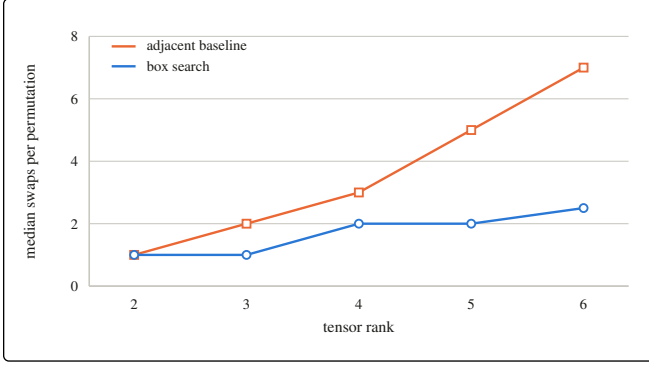
bytes moved is a difference in the decomposition rather than in the tensor.

Across the sweep the search issues 2.5x fewer swaps than the adjacent baseline and moves 2.4x fewer bytes at the median, reaching 5.8x on the permutation where it helps most. The margin widens with rank, as Figure 1 shows: at rank 6 the median permutation costs the baseline 7 swaps and the search 2.5. Table 1 gives the breakdown. 72% of the swaps the search selects bundle more than one axis into a box, which is the move the baseline cannot make and where the difference comes from.

The auxiliary cost of a plan is its cycle metadata, and it does not scale with the tensor: the largest plan in the sweep needs 3,488 bytes, or 2.16% of the data it permutes. The data itself is never copied to a second buffer.

3.3 Cost model

Edge weight is bytes moved divided by a calibrated throughput μ , currently 69.7 GB/s. Two thresholds gate the search: τ , the smallest D_{post} at which an edge is worth considering, and



References

Figure 1: **Median swaps per permutation against tensor rank.** 5 ranks, both planners on the same shapes. The baseline’s cost climbs with the number of adjacent transpositions a permutation needs; the search stays flat, ending at 2.5 swaps against 7 at rank 6.

D_{post}^* , the point at which box following beats a cycle splitter. Both currently sit at 1 and 1, so the pruning they describe is disabled.

[PLACEHOLDER] Derivation of the per-step cost, how μ is calibrated, and what τ and D_{post}^* prune once they are re-derived. Note also the gap between the documented cost and what the search actually minimises: during the search `estimated_cycle_info` reports no fixed points, so every edge weighs the same and the search minimises swap count rather than bytes. Exact counts are filled in afterwards, so the numbers above are accurate for the plans reported even though bytes were not what the search optimised.

4 Implementation

[PLACEHOLDER] The cycle-following kernel: scalar and float4 paths, the grid mapping, and the auxiliary cycle arrays.

5 Evaluation

[PLACEHOLDER] Setup, baselines, and results. Every number in this section must come from a generator under `paper/pipeline/`, reading a file under `raw_data/` that an `experiments/` script wrote. See `paper/pipeline/readme.md`.

6 Related Work

[PLACEHOLDER] Related work.

7 Conclusion

[PLACEHOLDER] Conclusion.